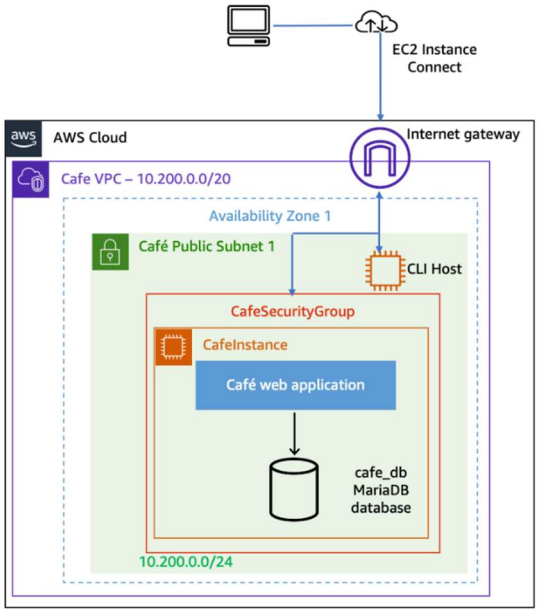
179--Lab - Migración a Amazon RDS

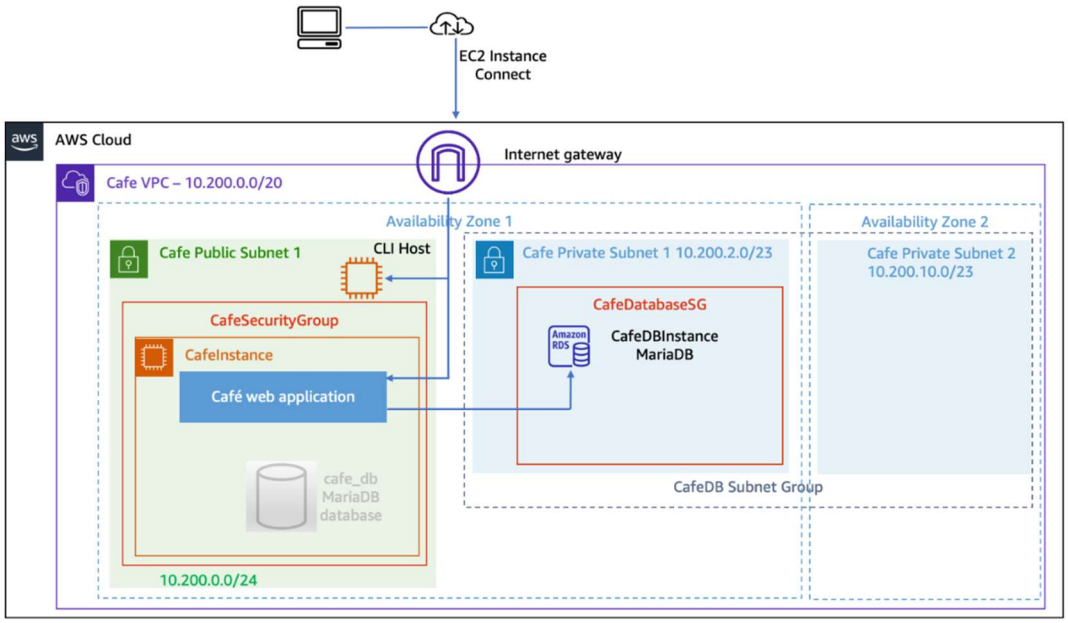
En esta práctica se muestra cómo migrar una base de datos MariaDB alojada en una instancia EC2 a una instancia RDS ubicada en una subred privada, y realizar los cambios en la aplicación web para apuntar a la nueva base de datos.

Escenario

La infraestructura actual es la siguiente:

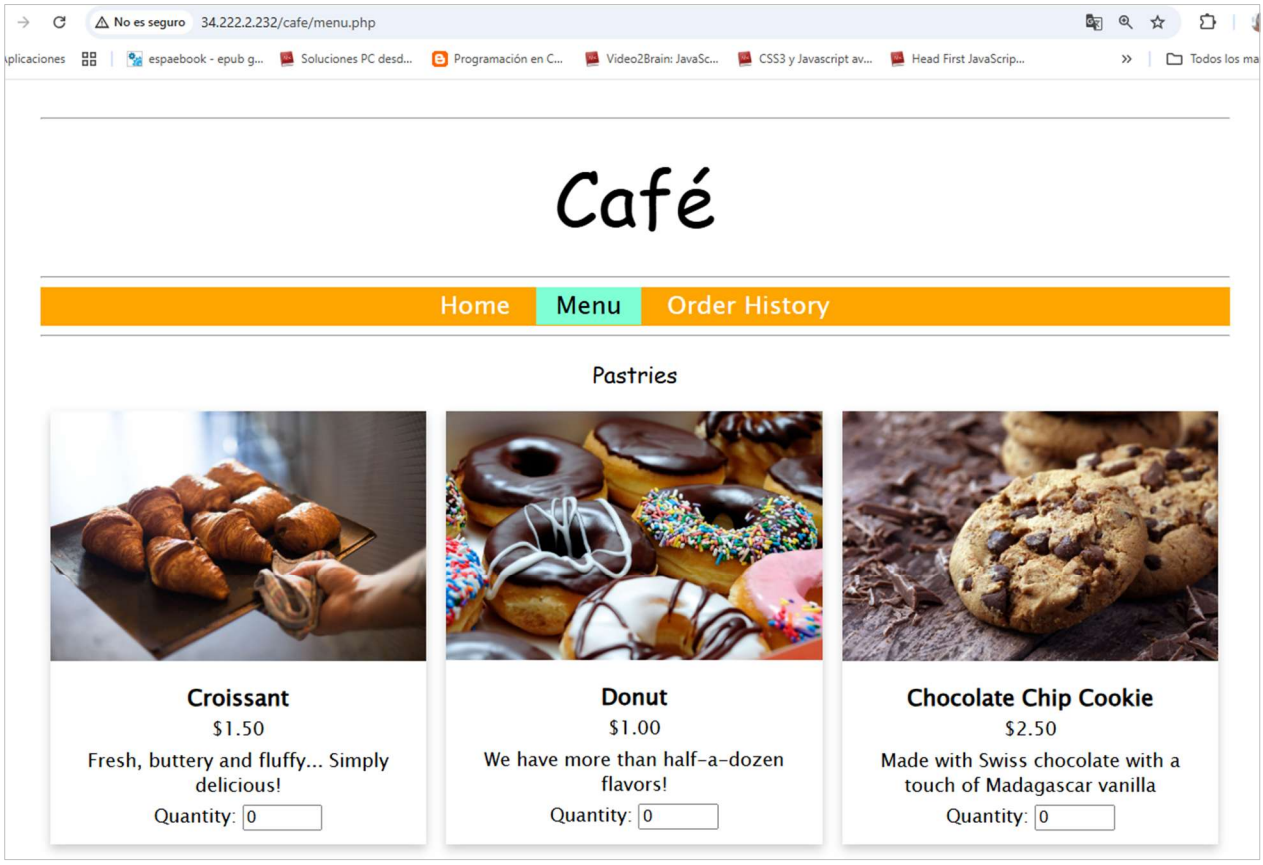


La infraestructura requerida es la siguiente:

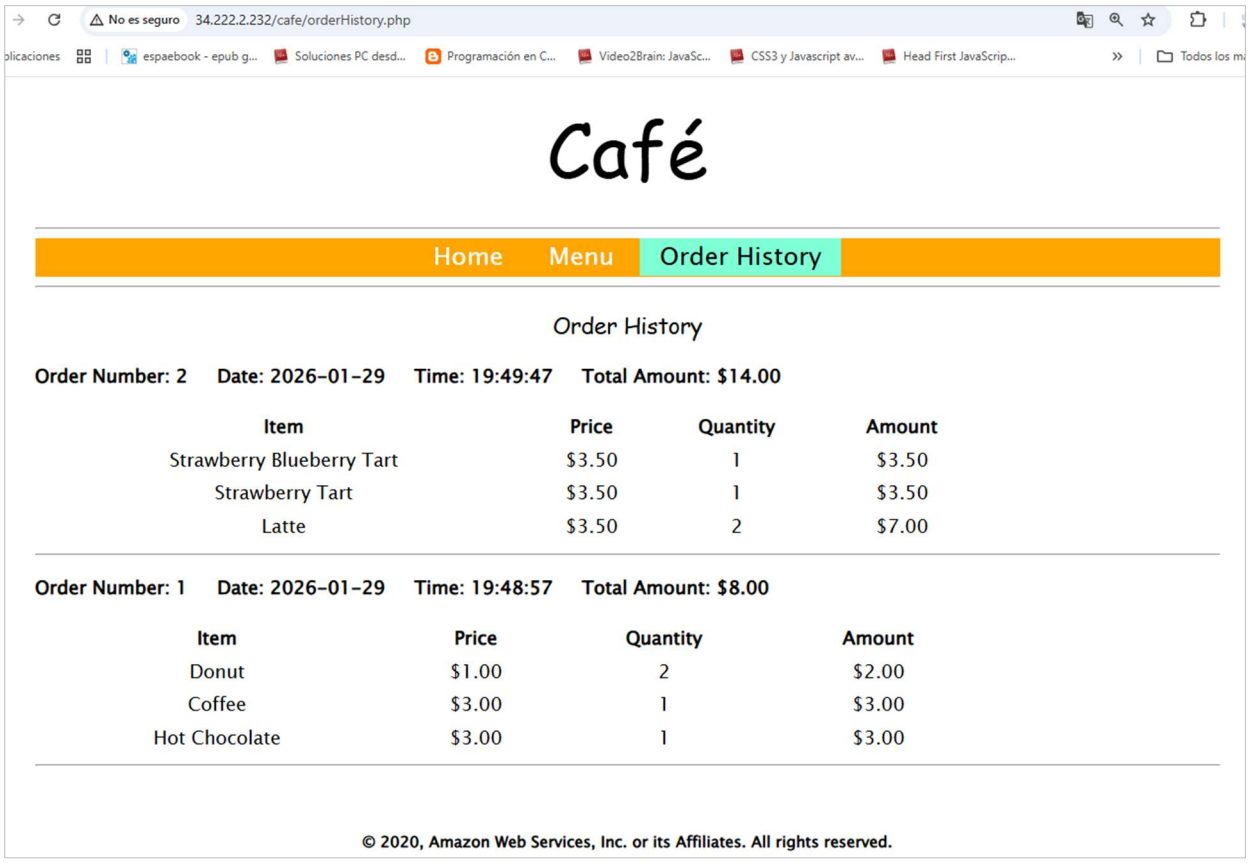


Generar datos en el sitio web de la cafetería

Para tener información en la base de datos a migrar, realizamos algunas órdenes en el sitio web

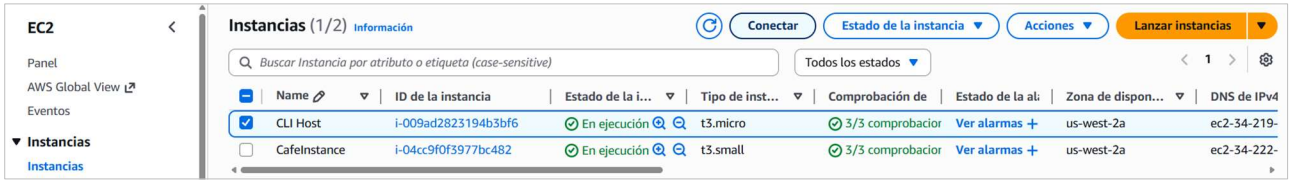


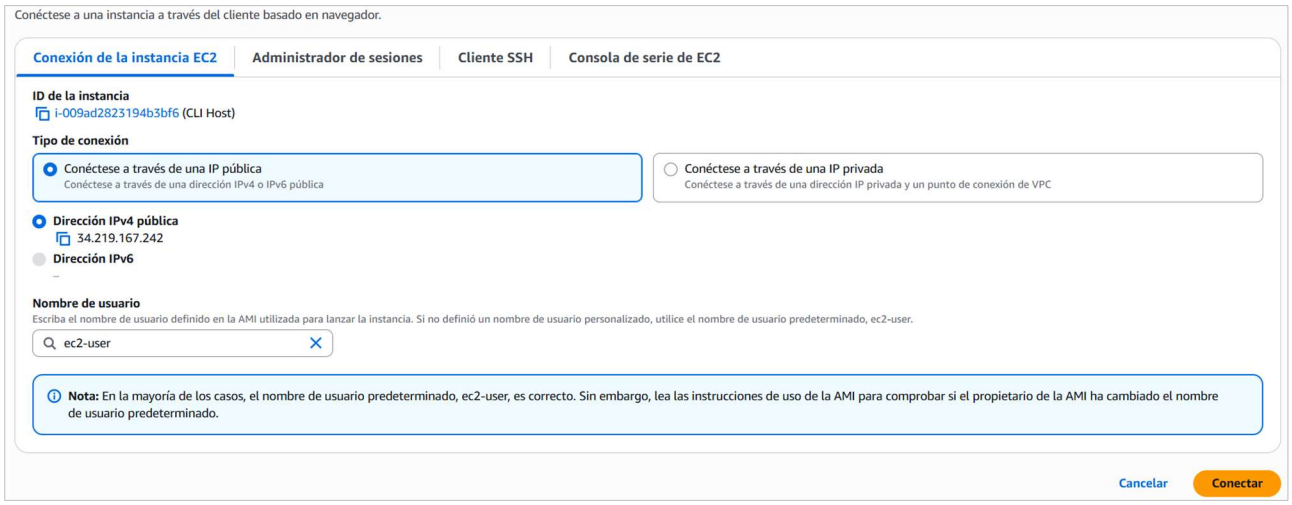
Checamos el historial de órdenes realizadas



Crear una instancia de Amazon RDS usando el AWS CLI

Accedemos a la terminal de la instancia CLI Host conectándonos por *Instance Connect*





Configuramos el AWS CLI introduciendo las claves de acceso y región

```
[ec2-user@ip-10-200-0-77 ~]$ aws configure
AWS Access Key ID [None]: AKIATT3GB5P567V5ON6D
AWS Secret Access Key [None]: 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
Default region name [None]: us-west-2
Default output format [None]: json
```

Para crear una instancia RDS desde el AWS CLI necesitamos lo siguiente:

- Un grupo de seguridad
- Subred privada 1
- Subred privada 2
- Grupo de subredes (subred privada 1 y subred privada 2)

Creamos el grupo de seguridad "CafeDatabaseSG"

```
aws ec2 create-security-group \
--group-name CafeDatabaseSG \
--description "Security group for Cafe database" \
--vpc-id vpc-03d8787006877b214
```

```
[ec2-user@ip-10-200-0-77 ~]$ aws ec2 create-security-group \
> --group-name CafeDatabaseSG \
> --description "Security group for Cafe database" \
> --vpc-id vpc-03d8787006877b214
{
    "GroupId": "sg-065090ac5d1de9202"
}
```

Creamos una regla de entrada en el grupo de seguridad para que permita el acceso, por el puerto 3306, al grupo de seguridad donde se encuentra la instancia EC2 con el sitio web de la cafetería

```
aws ec2 authorize-security-group-ingress \
--group-id sg-065090ac5d1de9202 \
--protocol tcp --port 3306 \
```

```
--source-group sg-0aee179252988d3cc
```

```
[ec2-user@ip-10-200-0-77 ~]$ aws ec2 authorize-security-group-ingress \
> --group-id sg-065090ac5d1de9202 \
> --protocol tcp --port 3306 \
> --source-group sg-0aee179252988d3cc
```

Ahora verificamos si se creó la regla de entrada solicitando el nombre del grupo (*GroupName*), el ID del grupo (*GroupId*) y los puertos que están abiertos y desde que direcciones IP se permite el tráfico (*IpPermissions*)

```
aws ec2 describe-security-groups \
--query "SecurityGroups[*].[GroupName,GroupId,IpPermissions]" \
--filters "Name=group-name,Values='CafeDatabaseSG'"
```

```
[ec2-user@ip-10-200-0-77 ~]$ aws ec2 describe-security-groups \
> --query "SecurityGroups[*].[GroupName,GroupId,IpPermissions]" \
> --filters "Name=group-name,Values='CafeDatabaseSG'"

[
    [
        "CafeDatabaseSG",
        "sg-065090ac5d1de9202",
        [
            {
                "PrefixListIds": [],
                "FromPort": 3306,
                "IpRanges": [],
                "ToPort": 3306,
                "IpProtocol": "tcp",
                "UserIdGroupPairs": [
                    {
                        "UserId": "248785267707",
                        "GroupId": "sg-0aee179252988d3cc"
                    }
                ],
                "Ipv6Ranges": []
            }
        ]
    ]
]
```

La regla de entrada se creó correctamente.

Ahora crearemos dos subredes privadas y un grupo de subredes.

Los bloques CIDR serán los siguientes:

	Bloque CIDR IPv4	**Rango**	
VPC	10.200.0.0/20	10.200.0.0	10.200.15.255
Subred privada 1	10.200.2.0/23	10.200.2.0	10.200.3.255
Subred privada 2	10.200.10.0/23	10.200.10.0	10.200.11.255

Creamos la primera subred privada en la zona de disponibilidad *us-west-2a*. En esta subred es donde estará la instancia RDS.

```
aws ec2 create-subnet \
--vpc-id vpc-03d8787006877b214 \
--cidr-block 10.200.2.0/23 \
--availability-zone us-west-2a
```

```
[ec2-user@ip-10-200-0-77 ~]$ aws ec2 create-subnet \
> --vpc-id vpc-03d8787006877b214 \
> --cidr-block 10.200.2.0/23 \
> --availability-zone us-west-2a
{
    "Subnet": {
        "MapPublicIpOnLaunch": false,
        "AvailabilityZoneId": "usw2-az1",
        "AvailableIpAddressCount": 507,
        "DefaultForAz": false,
        "SubnetArn": "arn:aws:ec2:us-west-2:248785267707:subnet/subnet-0121cdac7f612f169",
        "Ipv6CidrBlockAssociationSet": [],
        "VpcId": "vpc-03d8787006877b214",
        "MapCustomerOwnedIpOnLaunch": false,
        "AvailabilityZone": "us-west-2a",
        "SubnetId": "subnet-0121cdac7f612f169",
        "OwnerId": "248785267707",
        "CidrBlock": "10.200.2.0/23",
        "State": "available",
        "AssignIpv6AddressOnCreation": false
    }
}
```

Ahora creamos la segunda subred privada en la zona de disponibilidad *us-west-2b*

```
aws ec2 create-subnet \
--vpc-id vpc-03d8787006877b214 \
--cidr-block 10.200.10.0/23 \
--availability-zone us-west-2b
```

```
[ec2-user@ip-10-200-0-77 ~]$ aws ec2 create-subnet \
> --vpc-id vpc-03d8787006877b214 \
> --cidr-block 10.200.10.0/23 \
> --availability-zone us-west-2b
{
    "Subnet": {
        "MapPublicIpOnLaunch": false,
        "AvailabilityZoneId": "usw2-az2",
        "AvailableIpAddressCount": 507,
        "DefaultForAz": false,
        "SubnetArn": "arn:aws:ec2:us-west-2:248785267707:subnet/subnet-013e9177c9f984e91",
        "Ipv6CidrBlockAssociationSet": [],
        "VpcId": "vpc-03d8787006877b214",
        "MapCustomerOwnedIpOnLaunch": false,
        "AvailabilityZone": "us-west-2b",
        "SubnetId": "subnet-013e9177c9f984e91",
        "OwnerId": "248785267707",
        "CidrBlock": "10.200.10.0/23",
        "State": "available",
        "AssignIpv6AddressOnCreation": false
    }
}
```

Posteriormente creamos el grupo de subredes, indicando el ID de cada subred privada que creamos

```
aws rds create-db-subnet-group \
--db-subnet-group-name "CafeDB Subnet Group" \
--db-subnet-group-description "DB subnet group for Cafe" \
--subnet-ids subnet-0121cdac7f612f169 subnet-013e9177c9f984e91 \
--tags "Key=Name,Value= CafeDatabaseSubnetGroup"
```

```
[ec2-user@ip-10-200-0-77 ~]$ aws rds create-db-subnet-group \
> --db-subnet-group-name "CafeDB Subnet Group" \
> --db-subnet-group-description "DB subnet group for Cafe" \
> --subnet-ids subnet-0121cdac7f612f169 subnet-013e9177c9f984e91 \
> --tags "Key=Name,Value= CafeDatabaseSubnetGroup"
```


```
{
    "DBSubnetGroup": {
        "Subnets": [
            {
                "SubnetStatus": "Active",
                "SubnetIdentifier": "subnet-0121cdac7f612f169",
                "SubnetOutpost": {},
                "SubnetAvailabilityZone": {
                    "Name": "us-west-2a"
                }
            },
            {
                "SubnetStatus": "Active",
                "SubnetIdentifier": "subnet-013e9177c9f984e91",
                "SubnetOutpost": {},
                "SubnetAvailabilityZone": {
                    "Name": "us-west-2b"
                }
            }
        ],
        "VpcId": "vpc-03d8787006877b214",
        "DBSubnetGroupDescription": "DB subnet group for Cafe",
        "SubnetGroupStatus": "Complete",
        "DBSubnetGroupArn": "arn:aws:rds:us-west-2:248785267707:subgrp:cafedb subnet group",
        "DBSubnetGroupName": "cafedb subnet group"
    }
}
```


Teniendo todos los elementos necesarios ahora podemos crear la instancia RDS con los siguientes datos:

- DB instance identifier: CafeDBInstance
- Engine option: MariaDB
- DB engine version: 10.5.27
- DB instance class: db.t3.micro
- Allocated storage: 20 GB
- Availability Zone: us-west-2a
- DB Subnet group: CafeDB Subnet Group
- VPC security groups: CafeDatabaseSG
- Public accessibility: No
- Username: root

- Password: Re:Start!9

Primero verificamos cuales son las versiones de MariaDB disponibles para las instancias RDS

```
[ec2-user@ip-10-200-0-77 ~]$ aws rds describe-db-engine-versions --engine mariadb --query "DBEngineVersions[].EngineVersion"
[
    "10.5.27",
    "10.5.28",
    "10.5.29",
    "10.6.19",
    "10.6.20",
    "10.6.21",
    "10.6.22",
    "10.6.23",
    "10.6.24",
    "10.11.9",
    "10.11.10",
    "10.11.11",
    "10.11.13",
    "10.11.14",
    "10.11.15",
    "11.4.3",
    "11.4.4",
    "11.4.5",
    "11.4.7",
    "11.4.8",
    "11.4.9",
    "11.8.3",
    "11.8.5"
]
```

Usaremos la versión 10.5.27

```
aws rds create-db-instance \
--db-instance-identifier CafeDBInstance \
--engine mariadb \
--engine-version 10.5.27 \
--db-instance-class db.t3.micro \
--allocated-storage 20 \
--availability-zone us-west-2a \
--db-subnet-group-name "CafeDB Subnet Group" \
--vpc-security-group-ids sg-065090ac5d1de9202 \
--no-publicly-accessible \
--master-username root --master-user-password 'Re:Start!9'
```

```
[ec2-user@ip-10-200-0-77 ~]$ aws rds create-db-instance \
> --db-instance-identifier CafeDBInstance \
> --engine mariadb \
> --engine-version 10.5.27 \
> --db-instance-class db.t3.micro \
> --allocated-storage 20 \
> --availability-zone us-west-2a \
> --db-subnet-group-name "CafeDB Subnet Group" \
> --vpc-security-group-ids sg-065090ac5d1de9202 \
> --no-publicly-accessible \
> --master-username root --master-user-password 'Re:Start!9'
{
    "DBInstance": {
        "PubliclyAccessible": false,
        "MasterUsername": "root",
        "MonitoringInterval": 0,
        "LicenseModel": "general-public-license",
        "VpcSecurityGroups": [
            {
                "Status": "active",
                "VpcSecurityGroupId": "sg-065090ac5d1de9202"
            }
        ],
        "CopyTagsToSnapshot": false,
        "OptionGroupMemberships": [
            {
                "Status": "in-sync",
                "OptionGroupName": "default:mariadb-10-5"
            }
        ],
        "PendingModifiedValues": {
            "MasterUserPassword": "****"
        },
        "Engine": "mariadb",
        "MultiAZ": false,
        "DBSecurityGroups": [],
        "DBParameterGroups": [
            {
                "DBParameterGroupName": "default.mariadb10.5",
                "ParameterApplyStatus": "in-sync"
            }
        ],
        "PerformanceInsightsEnabled": false,
        "AutoMinorVersionUpgrade": true,
        "PreferredBackupWindow": "08:06-08:36",
```

```
        "DBSubnetGroup": {
            "Subnets": [
                {
                    "SubnetStatus": "Active",
                    "SubnetIdentifier": "subnet-0121cdac7f612f169",
                    "SubnetOutpost": {},
                    "SubnetAvailabilityZone": {
                        "Name": "us-west-2a"
                    }
                },
                {
                    "SubnetStatus": "Active",
                    "SubnetIdentifier": "subnet-013e9177c9f984e91",
                    "SubnetOutpost": {},
                    "SubnetAvailabilityZone": {
                        "Name": "us-west-2b"
                    }
                }
            ],
            "DBSubnetGroupName": "cafedb subnet group",
            "VpcId": "vpc-03d8787006877b214",
            "DBSubnetGroupDescription": "DB subnet group for Cafe",
            "SubnetGroupStatus": "Complete"
        },
        "ReadReplicaDBInstanceIdentifiers": [],
        "AllocatedStorage": 20,
        "DBInstanceArn": "arn:aws:rds:us-west-2:248785267707:db:cafedbinstance",
        "BackupRetentionPeriod": 1,
        "PreferredMaintenanceWindow": "tue:09:58-tue:10:28",
        "DBInstanceStatus": "creating",
        "IAMDatabaseAuthenticationEnabled": false,
        "EngineVersion": "10.5.27",
        "DeletionProtection": false,
        "AvailabilityZone": "us-west-2a",
        "DomainMemberships": [],
        "StorageType": "gp2",
        "DbiResourceId": "db-5OPYI6MTZUUGTZICK5HW6GYSFE",
        "CACertificateIdentifier": "rds-ca-rsa2048-g1",
        "StorageEncrypted": false,
        "AssociatedRoles": [],
        "DBInstanceClass": "db.t3.micro",
        "DbInstancePort": 0,
        "DBInstanceIdentifier": "cafedbinstance"
    }
}
```


La razón por la que "MultiAZ" es false en el JSON, a pesar de haber creado un grupo de subredes en zonas de disponibilidad distintas, es porque le dimos permiso a AWS de ejecutar la base de datos en cualquiera de las dos áreas, pero no incluimos el parámetro *–multi-az*. Por defecto, AWS crea una instancia única en una sola zona para ahorrar costos.

El "false" significa que la base de datos es una Instancia Única (Single-AZ). Aunque tiene "permiso" de estar en varias zonas gracias al grupo de subredes, actualmente solo vive en una. Si esa zona de disponibilidad falla, la base de datos se apagará porque no tiene una "réplica de reserva" (Standby) en la otra zona.

Single-AZ es una sola base de datos activa en una zona. Con *Multi-AZ* AWS crea automáticamente una segunda base de datos idéntica en la otra zona de tu grupo de subredes. Si la primera falla, la segunda toma el mando en segundos.

Ahora checamos el estatus de la base de datos

```
aws rds describe-db-instances \
--db-instance-identifier CafeDBInstance \
--query
"DBInstances[*].[Endpoint.Address,AvailabilityZone,PreferredBackupWindow,Backu
pRetentionPeriod,DBInstanceStatus]"
```

Con este query obtenemos la siguiente información de la instancia RDS:

- Endpoint address
- Availability Zone
- Preferred backup window
- Backup retention period
- Status of the database

```
[ec2-user@ip-10-200-0-77 ~]$ aws rds describe-db-instances \
> --db-instance-identifier CafeDBInstance \
> --query "DBInstances[*].[Endpoint.Address,AvailabilityZone,PreferredBackupWindow,BackupRetentionPeriod,DBInstanceStatus]"
[
    [
        "cafedbinstance.cl5mmlalk4lk.us-west-2.rds.amazonaws.com",
        "us-west-2a",
        "08:06-08:36",
        1,
        "backing-up"
    ]
]
```

El periodo de retención del respaldo es de 1 día

El estado es "backing up", es decir, está realizando una copia de seguridad de la base de datos

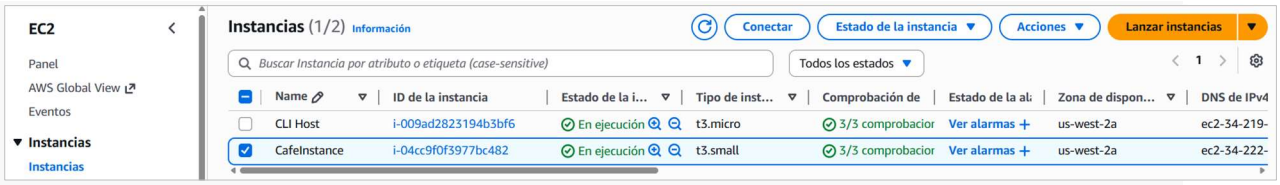
Checamos nuevamente el estatus

```
[ec2-user@ip-10-200-0-77 ~]$ aws rds describe-db-instances \
> --db-instance-identifier CafeDBInstance \
> --query "DBInstances[*].[Endpoint.Address,AvailabilityZone,PreferredBackupWindow,BackupRetentionPeriod,DBInstanceStatus]"
[
    [
        "cafedbinstance.cl5mmlalk4lk.us-west-2.rds.amazonaws.com",
        "us-west-2a",
        "08:06-08:36",
        1,
        "available"
    ]
]
```

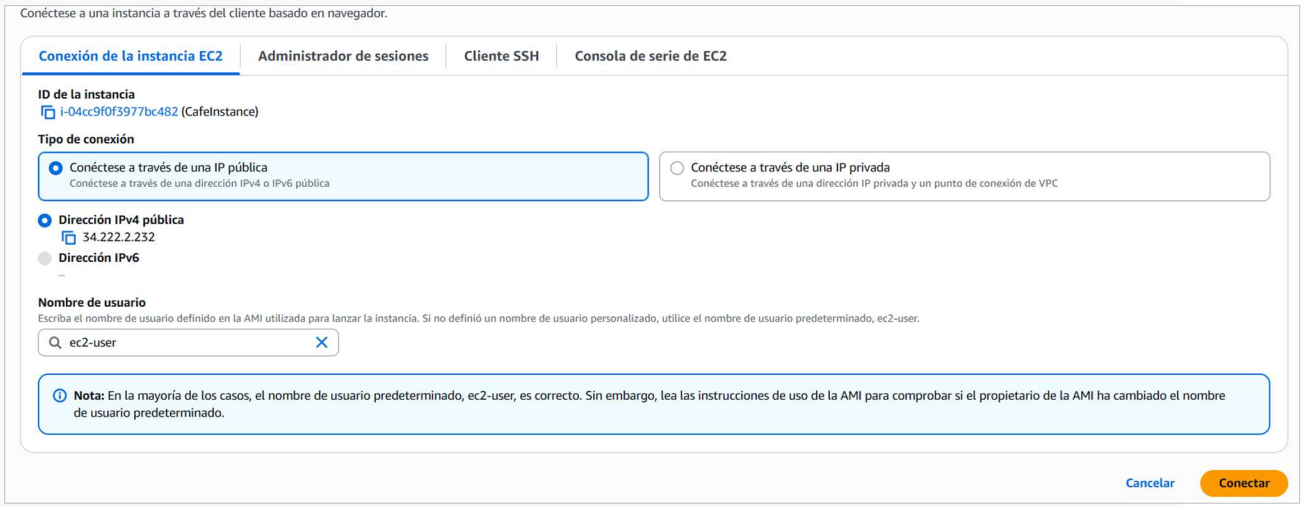
Ahora está disponible

Migrar los datos de la aplicación a la instancia RDS

Ahora nos conectamos a la instancia *CafeInstance*, que es la instancia donde está alojado el sitio web y la base de datos original



Nos conectamos por *Instance Connect*



Usaremos *mysqldump* para crear un respaldo de la base de datos alojada en la instancia.

Mysqldump es una utilidad de respaldo que convierte la base de datos en un archivo de texto lleno de instrucciones SQL (CREATE TABLE, INSERT INTO, etc.). Es el estándar para mover datos en MariaDB y MySQL.

```
mysqldump --user=root --password='Re:Start!9' \
--databases cafe_db --add-drop-database > cafedb-backup.sql
```

```
[ec2-user@ip-10-200-0-121 ~]$ mysqldump --user=root --password='Re:Start!9' \
> --databases cafe_db --add-drop-database > cafedb-backup.sql
```

```
[ec2-user@ip-10-200-0-121 ~]$ ls
cafedb-backup.sql
```

El parámetro *--add-drop-database* agrega una línea al archivo que dice: "Si ya existe una base de datos llamada *cafe_db* donde vas a importar esto, bórrala primero y luego crea la nueva".

Ahora creamos la base de datos *cafe_db* en la instancia RDS

```
mysql --user=root --password='Re:Start!9' \
> --host=cafedbinstance.cl5mmlalk4lk.us-west-2.rds.amazonaws.com \
> --execute="CREATE DATABASE cafe_db;"
```

```
[ec2-user@ip-10-200-0-121 ~]$ mysql --user=root --password='Re:Start!9' \
> --host=cafedbinstance.cl5mmlalk4lk.us-west-2.rds.amazonaws.com \
> --execute="CREATE DATABASE cafe_db;"
```

La base de datos ha sido creada.

Ahora ejecutamos el respaldo en la instancia RDS, para crear el esquema y los datos

mysql --user=root --password='Re:Start!9' \
--host= cafedbinstance.cl5mmlalk4lk.us-west-2.rds.amazonaws.com \
< cafedb-backup.sql

```
[ec2-user@ip-10-200-0-121 ~]$ mysql --user=root --password='Re:Start!9' \
> --host=cafedbinstance.cl5mmlalk4lk.us-west-2.rds.amazonaws.com \
> cafe_db < cafedb-backup.sql
```

Nos conectamos a la base de datos *café_db* en la instancia RDS

mysql --user=root --password='Re:Start!9' \
--host=cafedbinstance.cl5mmlalk4lk.us-west-2.rds.amazonaws.com \
cafe_db

```
[ec2-user@ip-10-200-0-121 ~]$ mysql --user=root --password='Re:Start!9' \
> --host=cafedbinstance.cl5mmlalk4lk.us-west-2.rds.amazonaws.com \
> cafe_db
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 78
Server version: 10.5.27-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [cafe_db]>
```

Hacemos una consulta a la tabla *product*

```
MariaDB [cafe_db]> select * from product;
+----+-------------------------+-----------------------------------------------------------------+-------+---------------+-----------------------------------------+
| id | product_name            | description                                                     | price | product_group | image_url                               |
+----+-------------------------+-----------------------------------------------------------------+-------+---------------+-----------------------------------------+
|  1 | Croissant               | Fresh, buttery and fluffy... Simply delicious!                  |  1.50 |             1 | images/Croissants.jpg                   |
|  2 | Donut                   | We have more than half-a-dozen flavors!                         |  1.00 |             1 | images/Donuts.jpg                       |
|  3 | Chocolate Chip Cookie   | Made with Swiss chocolate with a touch of Madagascar vanilla    |  2.50 |             1 | images/Chocolate-Chip-Cookies.jpg       |
|  4 | Muffin                  | Banana bread, blueberry, cranberry or apple                     |  3.00 |             1 | images/Muffins.jpg                      |
|  5 | Strawberry Blueberry Tart | Bursting with the taste and aroma of fresh fruit                |  3.50 |             1 | images/Strawberry-Blueberry-Tarts.jpg   |
|  6 | Strawberry Tart         | Made with fresh ripe strawberries and a delicious whipped cream |  3.50 |             1 | images/Strawberry-Tarts.jpg             |
|  7 | Coffee                  | Freshly-ground black or blended Columbian coffee                |  3.00 |             2 | images/Coffee.jpg                       |
|  8 | Hot Chocolate           | Rich and creamy, and made with real chocolate                   |  3.00 |             2 | images/Cup-of-Hot-Chocolate.jpg         |
|  9 | Latte                   | Offered hot or cold and in various delicious flavors            |  3.50 |             2 | images/Latte.jpg                        |
+----+-------------------------+-----------------------------------------------------------------+-------+---------------+-----------------------------------------+
9 rows in set (0.00 sec)
```

Hacemos una consulta a la tabla *order*

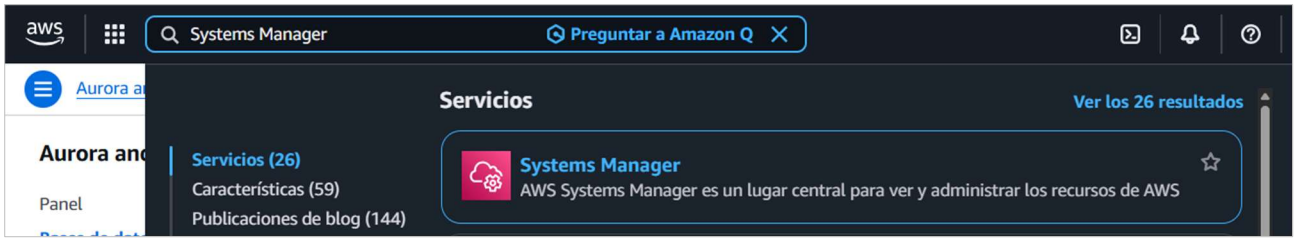
```
MariaDB [cafe_db]> SELECT * FROM `order`;
+--------------+---------------------+--------+
| order_number | order_date_time     | amount |
+--------------+---------------------+--------+
|            1 | 2026-01-29 19:48:57 |   8.00 |
|            2 | 2026-01-29 19:49:47 |  14.00 |
+--------------+---------------------+--------+
2 rows in set (0.00 sec)
```

Configurar el sitio web para usar la instancia RDS

El sitio web toma la cadena de conexión de *Parameter Store* en *AWS Systems Manager*

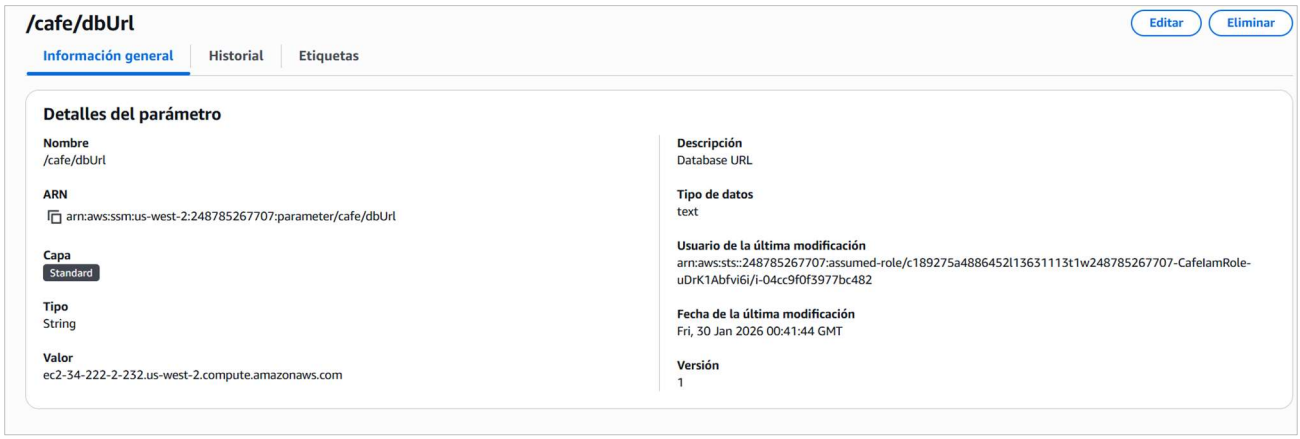
Modificaremos este valor para que apunta a la nueva instancia *RDS*

Accedemos al panel de *Systems Manager*



En *Parameter Store* seleccionamos el parámetro */cafe/dbUrl*

Hacemos clic en *Editar*



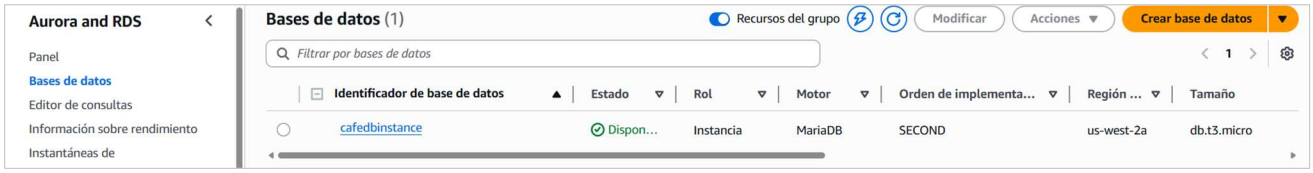
Modificamos el valor con el endpoint de la instancia RDS

Hacemos clic en *Guardar los cambios*

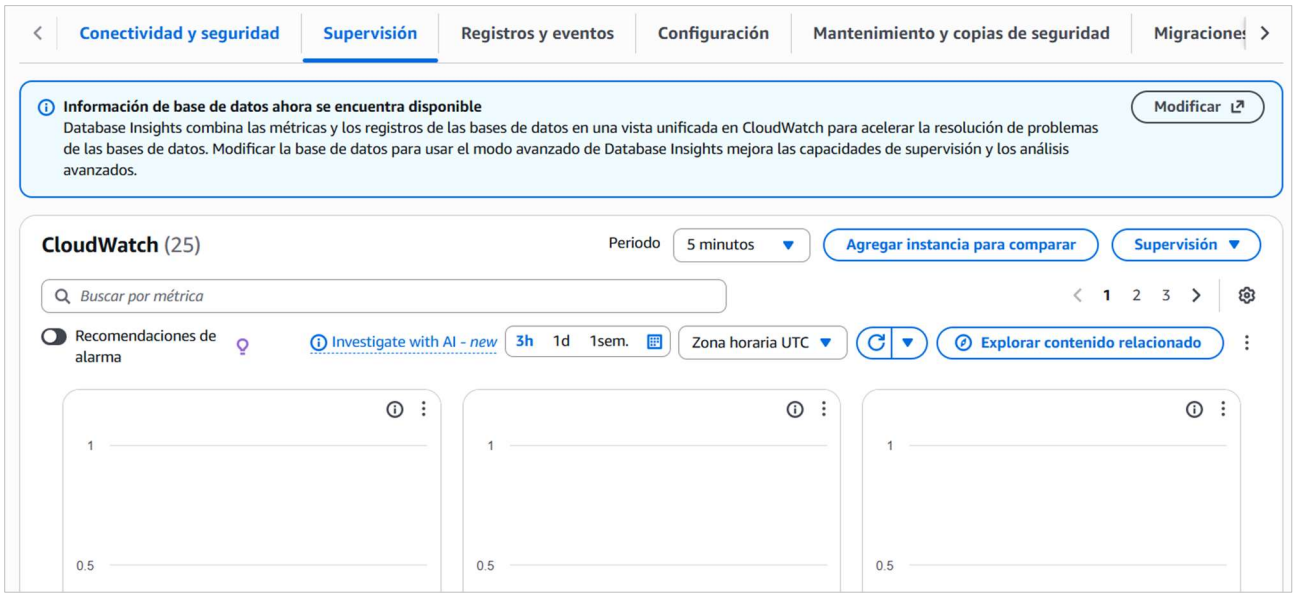
Monitorear la instancia RDS

Accedemos al panel de *Aurora and RDS*

En *Bases de datos* seleccionamos la base de datos *cafedbinstance*



En la pestaña *Supervisión* podemos ver métricas que *CloudWatch* monitorea con sus respectivas gráficas

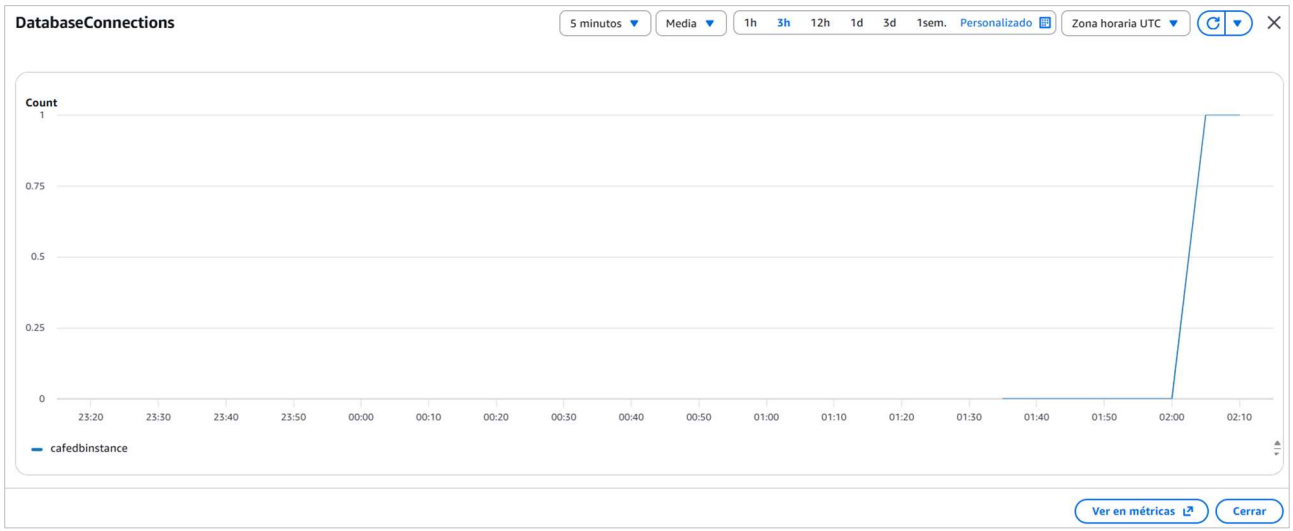


La lista de métricas que se muestran incluye las siguientes:

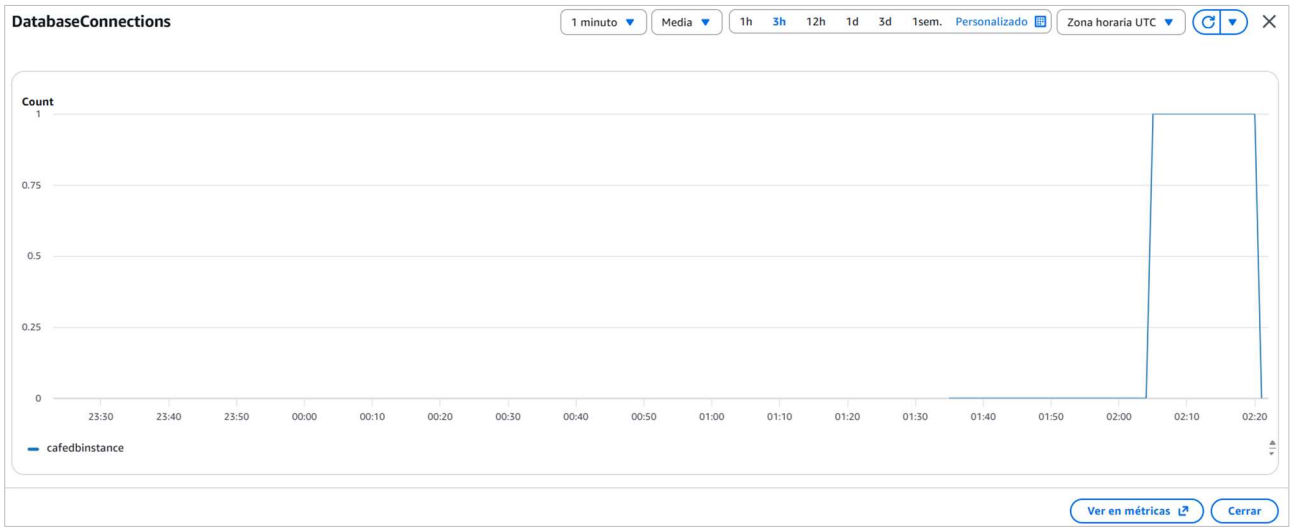
- **CPUUtilization:** el porcentaje de utilización de CPU
- **DatabaseConnections:** el número de conexiones de la base de datos en uso
- **FreeStorageSpace:** la cantidad de espacio de almacenamiento disponible
- **FreeableMemory:** la cantidad de memoria (RAM) disponible en la instancia de Amazon RDS
- **WriteIOPS:** el número promedio de operaciones de E/S de escritura en disco por segundo
- **ReadIOPS:** el número promedio de operaciones de E/S de lectura de disco por segundo

Podemos ver la métrica *DatabaseConnections*

Muestra un 1 porque hay una conexión en uso, que es la sesión de SQL que abrimos desde la consola



Salimos de la consola de MariaDB

```
MariaDB [cafe_db]> exit
Bye
```

Podemos ver que la métrica bajó a 0